

oa-tools - A Next-Generation Remastering Suite

After years of passionate work on **penguins-eggs**, I am now undertaking its next evolution: **oa-tools**. **oa-tools** is built on **oa**, a high-performance C-native remastering engine, and **coa**, its intelligent Go orchestrator. This new architecture replaces fragile bash scripts with native Linux kernel syscalls and strict zero-copy principles — delivering a significant leap in speed and reliability. The goal is ambitious: a truly universal approach to Linux remastering. Every distribution is different, yet they all share common roots. **oa-tools** already supports Alpine, Arch, Debian, Fedora, Manjaro and openSUSE — and aims to prove that a single, unified engine can handle them all.



Changelog - oa-tools

Release Notes: oa-tools v0.8.6 - "The Logic Tractor" 2026-06-10

Overview

This release marks a fundamental shift in the development lifecycle of **oa-tools**. We have transitioned from a monolithic, script-based approach to a highly orchestrated, multi-layered architecture. By separating high-level planning from mechanical execution, we have transformed the build process into a robust, predictable pipeline.

The Architectural Shift

In developing this version, we embraced a design philosophy rooted in the separation of concerns:

- **The Declarative Layer (JSON/Plan):** We now define *what* the system should achieve through a declarative JSON-based plan. This is the "knowledge" of the build—a high-level map that remains agnostic of the underlying implementation details.
- **The Orchestration Layer (Go):** This acts as the "Logic Tractor." It parses the high-level plan and translates it into specific, deterministic actions. By isolating the logic from the mechanics, we have eliminated the fragility of tightly coupled scripts, ensuring that the control flow remains consistent regardless of the environment.
- **The Mechanical Layer (Bash/C):** This layer handles the "heavy lifting." By delegating the execution to focused, modular Bash scripts, we have replaced complex inline templating with clean, testable, and reliable procedures.

Key Technical Achievements

- **Decoupled Orchestration:** Moving from hardcoded logic to a modular, plan-driven execution model has drastically reduced build failures and increased system observability.
- **Universal Bootloader Logic:** We have standardized the bootloader installation process. By injecting a unified, self-contained installation script (**oa-bootloader.sh**) into the target

chroot, we successfully resolved the long-standing cross-compatibility issues between BIOS/Legacy and UEFI systems.

- **Atomic Cleanup:** Through the integration of the final cleanup phase directly into the deployment logic, we have ensured that no live artifacts persist in the installed environment. The system is now as clean as a fresh, official distribution install, with zero manual overhead.

Reflection

Building this system has been an exercise in structural engineering. Just as modern intelligent systems rely on the separation of data, logic, and execution, `oa-tools` now follows these fundamental laws. The "Logic Tractor" no longer struggles with the "hoe"—the separation of the orchestrator from the machinery has granted us a level of scalability and stability that was previously unattainable.

This release is not just a collection of bug fixes; it is the foundation for a professional-grade build infrastructure.

Release v0.8.5: The Declarative Evolution 🚀 2026-06-06

This release marks a fundamental architectural milestone for the framework. We have successfully transitioned from a heavily imperative Bash-scripted workflow to a clean, highly robust **declarative YAML orchestration** model.

The engine is now smarter, the codebase is significantly lighter, and the remastering flight plan is easier to read, maintain, and extend.

🌟 Key Architectural Changes

- **The Declarative Engine (`base.yaml.tpl`):** The entire live ISO generation process is now orchestrated by a master YAML plan. This provides an Ansible-like, step-by-step clear overview of the remastering and installation hooks.
- **The `oa_ell` Bridge:** Upgraded the internal Go worker (`cmd/ell.go`) to act as a flawless bridge between the C engine and the system. It features a new, indestructible JSON router capable of seamlessly handling tasks like dynamic templating, file copying, and native shell execution.
- **Massive Bash Reduction:** Hundreds of lines of complex, imperative Bash logic have been retired. Core configurations—including bootloader staging (GRUB/ISOLINUX), sudoers setup, trusted desktop launchers, universal autologin, and SVG icon generation—are now handled purely through declarative `ell-actions.tpl` tasks.

🔧 Core Improvements & Fixes

- **Unified Squashfs Compression:** Streamlined the root filesystem compression logic (`core_squashfs`). Dropped legacy container-build workarounds in favor of a highly optimized, dynamically injected local profile (defaulting to multi-core `zstd` with automatic fallback configurations).
- **Bulletproof YAML Parsing:** Resolved deep-level scope and Go-template injection edge-cases (`{{- include ... }}`). The `pilot` engine now generates the exact JSON flight plan

flawlessly, immune to invisible characters or indentation mismatches.

- **Library Standardization:** Reorganized the template libraries with clear roles and strict 80-column headers (`base.yaml.tpl`, `scripts.tpl`, `ell-actions.tpl`), establishing a clean standard for future open-source contributions.
- **Smarter User Identity Handling:** The internal worker now reliably maps and sanitizes user data, executing native identity injection (e.g., creating the `live` user with specific groups and GID/UID) without dropping to shell scripts.

🎯 What's Next?

With the YAML/Go/C orchestration bridge now fully operational and stable, the framework's foundation is rock-solid. Adding new features or supporting new distributions is now just a matter of adding a few declarative lines to the YAML plan.

[0.8.4] - 2026-06-03

Added

- **New universal `coa tools grub40` command:** Introduced an advanced feature to automate loopback booting of any Linux ISO directly from the host's GRUB menu, eliminating the need for USB flashing during remastering tests.
 - **Ultra-fast inspection via `bsdtar`:** Switched from `xorriso` to native `bsdtar` parsing. The tool now dissects the target ISO in milliseconds, extracting the real paths of Kernel and Initrd from internal configuration files (`grub.cfg`, `isolinux.cfg`).
 - **Dynamic handling for Arch Linux (Kiro/Archiso):** Resolved the recovery shell boot hang (`device did not show up`) by injecting the GRUB `probe` module. GRUB dynamically calculates the host partition's UUID at runtime, passing it to Archiso via `img_dev` and `img_loop` parameters.
 - **Native Debian/Ubuntu integration:** Seamless support for the `live-boot` engine by automatically mapping the `findiso` boot parameter.
 - **Automation with `--write` / `-w` flag:** Added the ability to directly inject the generated `menuentry` into `/etc/grub.d/40_custom`.
 - **Surgical replacement using markers:** The injection mechanism uses unique identifier comments based on the ISO filename (`# >>> oa-tools start...`). This automatically overwrites existing entries when rebuilding the same ISO, preventing file clutter while strictly preserving file execution permissions (`0755`).
 - **Smart host distro detection:** Automatically identifies the host operating system to suggest or run the correct bootloader update command (`update-grub` on Debian/Ubuntu, `grub-mkconfig` on Arch/Fedora).

[0.8.3] - 2026-06-01

Added

- **The Furnace CI:** Fully automated Proxmox/GitHub Actions pipeline for unattended ISO generation.
- Support for automated build matrices across Arch, Debian, Alpine, and Fedora.

- Dynamic IP discovery in CI via targeted ARP sweep (bypassing static MAC/IP configurations).
- `repo add` and `repo rm` commands for cross-distribution repository management.
- Bugfix: Resolved an issue where the ISO dropped to (initramfs) in VirtualBox by implementing the proper `.disk/info` and `.disk/id` metadata for Debian live-boot.
- Build: Cleaned up the release version naming in the Makefile by removing the commit hash from the generated tags.
- Added `coa tools grub40` command to automatically generate universal GRUB 40_custom entries for booting ISOs via loopback.

Fixed

- Handled missing `/etc/skel` and BusyBox `cp` limitations during the creation of the live user home directory on Alpine Linux.
- Bypassed Linux kernel ping broadcast restrictions during automated CI network discovery.

Changed

- Improved SELinux compatibility during the remastering process for Fedora-based systems.

Deprecated / Beta

- `repo add/rm` commands are currently in Beta for openSUSE and Alpine Linux pending minor compatibility fixes.

oa-tools v0.8.2: Packaging fixes, dynamic exclusions & Config rollout

I believe these new features will be highly interesting for many users: the ability to change the live user password and customize the compression algorithm, as well as easily add or remove custom exclusions during the remastering process.

- **feat(config):** officially published the configuration file (`/etc/oa-tools.d/custom.yaml`, previously omitted by mistake). Users can now easily customize system-wide settings, including the live user password, as well as mksquashfs compression algorithms and parameters.
- **feat(engine):** revamped `/etc/oa-tools.d/custom.exclude.list` with robust parsing (elegantly ignoring comments and blank lines) and populated it with default exclusions for heavy container engines (Docker, Podman, LXC/LXD) and Snap. Users can easily disable these defaults or expand the list at will to keep their generated ISOs perfectly lean.

oa-tools v0.8.0: The Architectural Leap & Expanded Horizons

This is a fundamental release for `oa-tools`, introducing a rock-solid template architecture and extended support for new distributions.

✨ Key Features & Improvements

- **Bulletproof Template Architecture:** Introduced strict `core_` and `hook_` nomenclature for all templates. This eliminates silent "shadowing" conflicts and provides a highly

scalable foundation (ready for the upcoming Bianbu OS / RISC-V porting).

- **Alpine "Sidecar" Parachute:** Implemented a native recovery system in the initramfs for Alpine Linux, ensuring stable OverlayFS mounting and virtual filesystem relocation right before OpenRC boot.
- **openSUSE Support:** Added full support for openSUSE, bringing `oa-tools` on par with the historical parent distributions supported by `penguins-eggs`.
- **CLI & UX Enhancements (Cobra):** * Native shell auto-completion (Bash/Zsh/Fish) dynamically generated during `.deb` packaging for both `coa` and the legacy `eggs` alias.
 - New `coa export log` command to instantly extract debug logs to a remote host via SSH.
- **Semantic Polish:** Renamed the destructive command to `eggs destroy` to align with standard DevOps terminology.

`oa-tools` v0.7.9 - The CI/CD & Architecture Release

Evolution Architecture

- **Decoupled Engine & Pilot:** Completely refactored the Go architecture to eliminate import cycles. Environmental awareness (like detecting GitHub Actions) is now handled natively by the `cmd` package (Command Bridge), leaving `engine` and `pilot` strictly agnostic and modular.
- **Dynamic Template Injection:** The `TemplateContext` now dynamically receives environmental flags to adapt script generation on the fly without logic duplication.

CI/CD & Automation Breakthrough

- **GitHub Actions Turbo Mode:** Introduced automatic CI detection. When running on GitHub Actions, the system auto-injects a "Turbo" profile for `mksquashfs`, applying aggressive `zstd` compression and smart exclusions to slash ISO size (down to 6.4GB) and drastically reduce build times.

`oa-tools` v0.7.8 -

- **Smart Desktop Management:** Installation links are now dynamically handled and automatically removed after a successful installation, keeping the target `/etc/skel` clean.

Evolution Architecture

- **Modular Template Engine:** Decoupled YAML orchestration from Bash logic using a new "Helm-style" system.
- **Enhanced Bash Isolation:** Moved all scripts to dedicated `.bash.tpl` files with zero-margin indentation for better maintainability.

Technical Improvements

- **Cross-Distro Stability:** Verified seamless support for Debian, Arch, Fedora, and Manjaro using the new architecture.
- **Pilot Logic:** Added `include` and `indent` functions to the Go engine for perfect YAML syntax generation.

Release v0.7.7

Core Engine & Remastering

- **Fixed SquashFS Exclusions:** Corrected wildcard expansion rules (`.*`) in `mksquashfs` exclude lists that previously caused the unintended removal of vital directories (like `/mnt`, `/media`, and `/root`) during the live filesystem compression.

Live System & Templates

- **Universal Autologin:** Replaced distro-specific workarounds with a unified, robust LightDM autologin bypass. It successfully handles PAM restrictions and locked passwords across Debian, Arch Linux, and Fedora live sessions.
- **Trusted Desktop Launcher:** Implemented an automated background script ("Secret Agent") that dynamically applies GIO metadata and checksums to authorize the Calamares installer icon across all major Desktop Environments (XFCE, GNOME, KDE, Cinnamon, etc.) without security warnings.
- **Dynamic 3D SVG Icon:** The live system now dynamically generates a high-quality, 3D-styled SVG egg icon directly from the YAML template.

Calamares Installer

- **Slideshow Redesign:** Overhauled the `show.qml` presentation with a new "Arctic Gold" aesthetic, featuring improved typography and high-contrast text outlines for perfect readability over snow/bright backgrounds.
- **Responsive Image Scaling:** Introduced a dynamic mathematical container in QML. Slideshow images now scale perfectly to fit the installer window while locking the text inside the image boundaries, eliminating text overflow and blank borders.

Release v0.7.6: New Template Architecture** - 2026-05-10

This release marks a fundamental structural shift for `oa-tools`: the transition to a modular build system.

We have separated the system logic using templates:

- **Universal framework:** Actions common to all systems are now centralized in `brain.d/base.yaml.tpl`.
- **Specific modules:** Code unique to each distribution (Debian, Arch, Fedora, Manjaro) has been isolated within `brain.d/modules/`.

This approach drastically reduces code complexity compared to previous versions and simplifies readability. Beyond stabilizing our current work, this architectural cleanup provides the structural foundation necessary to explore new developments and support scenarios that were previously out of reach.

[0.7.5] - 2026-05-03

Added

- **Universal Boot Branding:** Integrated support for custom splash screens (`splash.png`) and Unicode fonts (`unicode.pf2`) for both BIOS (Isolinux) and UEFI (GRUB).
- **Enhanced GRUB Layout:** Added a dynamic title and visual spacing to the GRUB menu for a professional look.
- **Dynamic OS Detection:** Boot menus now automatically display the distribution name (via `$PRETTY_NAME`).
- **Comprehensive Shell Support:** Full completions for Bash, Zsh, and Fish, including automatic 'eggs' alias registration.
- **Fedora UI Fixes:** Added `google-noto-emoji-fonts` as a dependency in the RPM builder to ensure correct symbol rendering in the terminal.

Fixed

- **Fedora UEFI Visibility:** Added `efi_gop` and `efi_uga` modules to GRUB configuration to fix splash screen issues in UEFI mode.
- **Isolinux Compatibility:** Standardized the `APPEND` syntax for boot parameters, ensuring a successful boot on Fedora and other non-Debian systems even when using Debian bootloaders.
- **RPM Asset Packaging:** Updated the Go builder to correctly package and deploy branding assets into `/etc/oa-tools.d/brain.d/assets/`.

Technical Notes

- The "oa" dialect is now fully established for egg-based eggs-bananas remastering.
- All configurations are now centralized in the "brain" directory for better portability.

Release 0.7.4: The Great Refactoring

Fedora is now aligned to the others distros: arch, debian and manjaro.

This release marks a turning point for **oa-tools**. We have moved beyond chasing the specific quirks of individual distributions to build a universal, fluid, and frictionless infrastructure.

Architectural Evolution

The core has been surgically redesigned to separate concerns between the "Brain" and the "Muscle":

- **The Orchestrator (Go):** Now acts as the project's intelligence. It parses profiles, manages logic, and generates a perfect "flight plan" (`oa-plan.json`). Its predictive logic prevents failures before they even begin.
- **The Engine (C):** The `oa` binary is the operational arm. It consumes the JSON plan and interacts directly with the OS with the speed and precision of C, handling mounts, chroots, and identity injection in a secure, isolated environment.
- **Unified Dialect:** We have standardized how tasks are defined. The keys `action`, `description`, and `run_command` now drive every operation, making profiles significantly more readable and maintainable.

 Coming Soon: Profile Simplification

While this release solidifies the backend, we are already looking ahead. **The next step is a massive simplification of the profile structure.** By implementing **Go Templates**, we will drastically reduce boilerplate code, allowing a single profile to adapt dynamically to different environments and architectures without manual duplication.

 Technical Highlights

Feature	Description
Cross-Distro Validated	Full support for Debian (Trixie), Arch, Fedora, and Manjaro.
Robust Chroot Environment	Automatic PATH handling and <i>UsrMerge</i> link replication to ensure post-install success.
JSON-Driven Execution	Complete decoupling of task definition from operational execution.
Advanced Emergency Cleanup	Improved "nest" clearing to ensure the system returns to a clean state after every run.

[0.7.3] - 2026-05-01

Core Improvements & Fixes

Full compatibility achieved with Linux Mint, the Ubuntu family, and its derivatives.

[0.7.2] - 2026-05-01

Core Improvements & Fixes

- **Standardized Sudo Privileges:** Implemented a unified "Arch-style" sudo configuration for the live user, ensuring seamless operation of **oa** tools across Arch, Debian, and Manjaro bases.
- **Enhanced Build Cleanup:** Integrated a "tabula rasa" policy during the remastering process to automatically remove legacy or malformed configuration files (e.g., **00_artisan**) that previously caused syntax errors in the sudoers directory.
- **Post-Installation Optimization:** Added an automated cleanup routine to the **FINALIZE** stage. This ensures that the installed system is stripped of live-environment "ghost" files, leaving only the necessary installer-generated permissions.
- **Permission Hardening:** Enforced strict **0440** permissions for generated sudoers files to comply with security standards and prevent them from being ignored by the system.

This release marks a significant step in the **eggs-bananas** philosophy, delivering a cleaner, more professional transition from the live environment to the final installation.

[0.7.1] - 2026-04-30

"The Streamlined Artisan" Update

This minor release focuses on code purification and structural robustness, following our core "Eggs & Bananas" philosophy. We've removed redundant configuration files and refined the distro-agnostic engine for better performance and reliability.

Added

- **Smart Directory Management:** The Arch Linux and Manjaro builders now correctly install the `brain.d` logic directory into `/etc/oa-tools.d/`, ensuring agnostic mapping is available immediately after installation.
- **Enhanced Distro Detection:** Integration with system `ID_LIKE` metadata allows for seamless recognition of derivative distributions without external mapping files.

Changed

- **Code Purification:** Removed the `DistroUniqueID` field and the `derivatives.yaml` dependency. The system now relies on pure `/etc/os-release` data, making the codebase leaner and easier to maintain.
- **Refactored NewDistro:** Optimized the recognition logic to prioritize system-provided identity over manual overrides.

Fixed

- Fixed a missing directory copy in the Arch Linux `PKGBUILD` that prevented `coa` from finding its core logic.
- Improved error handling in the `distro` package: the system now enters a "Generic" mode instead of exiting when encountering an unmapped distribution.

Release Note: oa 0.7.1 - Simplicity is the Ultimate Sophistication

With **oa 0.7.1**, we continue to strip away the unnecessary. By moving the distribution mapping directly into the core logic and leveraging system metadata, we've eliminated the need for external configuration files.

This release ensures that your "Artisan" environment on Arch Linux is configured perfectly from the first `makepkg`. It's faster, cleaner, and more robust.

The artisan's tool just got sharper.

[0.7.0] - 2026-04-29

"The Agnostic Artisan" Release

This release implements the **"Eggs & Bananas"** philosophy: reducing maintenance costs by 90% while retaining the power of the original wardrobe. The tailor logic is now lighter, faster, and ready for a multi-distro world.

Added

- **Distro Agnosticism:** `oa` now detects the running distribution and adapts its "tailoring" strategy.
- **AI-Assisted Tailoring:** For non-Debian systems (like Arch Linux), `oa` now generates a comprehensive `AIPrompt.txt` directly in the user's Home directory.
- **Hardware-Aware Prompts:** The AI prompt now includes `lspci` data (VGA/3D controllers) and available `xsessions` to help the AI suggest the correct drivers and display manager configurations.
- **Smart APT Filtering:** On Debian-based systems, `oa` now verifies package existence via `apt-cache` before attempting installation, preventing a single missing package from breaking the entire process.
- **Robust Path Resolution:** Post-installation scripts (using `../..../scripts/` syntax) are now automatically resolved to absolute paths and granted execution permissions.
- **Persistent Logging:** Added `/var/log/coa-tailor.log` for background auditing and improved user-facing logs.

Changed

- **Flat YAML Structure:** Deprecated the nested `sequence/finalize` blocks in favor of a flat, clean `index.yaml` at the root of each costume/accessory.
- **Simplified Wardrobe:** Removed legacy repository management from the wardrobe logic to favor system-native tools or AI guidance.
- **Refactored Tailor Engine:** The `wear` logic is now 90% simpler to maintain while remaining 90% as convenient as the original implementation.

Fixed

- Fixed script execution errors where relative paths were not found by the shell.
- Fixed `sudo` environment issues: `AIPrompt.txt` and synced home files now correctly belong to the real user even when run with root privileges.

Release Note: oa 0.7.0 - The Agnostic Artisan

We are proud to announce the release of **oa 0.7.0**.

In the spirit of pragmatism, we have stripped away the complexity that made cross-distro support difficult. By embracing the "Eggs & Bananas" approach, we've created a "Tailor" that isn't just a package installer, but an intelligent assistant.

Whether you are on a "naked" Debian or a fresh Arch Linux installation, `oa` helps you dress your system with your favorite costumes. If it can't install a package automatically, it provides you with a "Medical Record" of your system (`AIPrompt.txt`) to hand over to an AI assistant, ensuring you get the perfect configuration for your specific hardware (including tricky ones like QXL or Nvidia).

It's time to sow the seeds of a truly universal Linux customizer. ` Ho sistemato il Changelog includendo tutti i punti chiave discussi. Buona semina per questa versione 0.7.0! 🐧



[0.6.5] - 2026-04-15



Added

- **"eggs" Alias Support:** Implemented dual binary identity. The package now installs symlinks and shell completion patches (Bash/Zsh/Fish) to allow using the `eggs` command as an alias for `coa`.
- **Standardized Naming Engine:** Isolated naming logic into `naming.go`. ISOs now follow the format: `egg-of_[distro]-[codename]-[hostname]_[arch]_[timestamp].iso`.



Changed

- **CLI Semantic Refactoring:** Renamed core commands for clarity: `produce` is now `remaster` and `krill` is now `sysinstall`.
- **Build System Decoupling:** Split the monolithic `build.go` into distribution-specific handlers (`build_debian.go`, `build_arch.go`, `build_fedora.go`) for granular dependency and metadata management.
- **Universal Boot Extraction:** Refactored boot file extraction to natively support Arch, Debian, and Fedora structures during the `remaster` phase.
- **Arch Linux Initrd Refactor:** Optimized ramdisk generation on Arch via deeper `mkinitcpio` integration.



Fixed

- **Fedora 43 Boot (BLS/EXT4):** Resolved "grub rescue" issues by forcing the deactivation of *Boot Loader Specification* (`GRUB_ENABLE_BLSCFG=false`) and applying EXT4 compatibility flags (`^metadata_csum_seed`, `^orphan_file`) during formatting.
- **Installer Hang (Sync & Cleanup):** Fixed a deadlock during the final Krill phase on Fedora by making `/var/lib/live` cleanup selective for Debian and introducing a forced `sync` before unmounting partitions.
- **Chroot Pathing:** Fixed path resolution errors during `oa_shell` execution in chroot environments, ensuring correct injection of hostname and sudoers.

Packaging

- **RPM Dependency Mapping:** Corrected Fedora RPM metadata by mapping `sgdisk` requirements to the `gdisk` package.
- **Symlink Integration:** All native packages now include symlinks for binaries, man pages, and shell completions for the `eggs` alias.

[0.6.1] - 2026-04-10

Fixed

- **Arch Linux Live Boot:** Resolved a critical issue causing Kernel Panic on boot. The ramdisk generation now correctly includes the `archiso` hook.
- **Initramfs Chroot Enforcement:** Fixed a bug where `mkinitcpio` and `mkinitramfs` were reading host system configurations instead of the target `liveroot` configurations. Commands are now explicitly forced to use `-c` or `-d` flags pointing to the chroot.

- **Bootloaders Tarball Extraction:** Restored the directory flattening logic (`parts[1:]`) in the `extractTarGz` utility to prevent nested root folders when downloading from GitHub releases.
- **Engine Logs:** Suppressed false-positive `cp` error outputs (`stderr`) in the C engine (`lay_livestruct.c`) during fallback kernel extraction attempts, resulting in cleaner build logs.

Changed

- **Universal Bootloaders Path:** Unified the `BootloadersPath` assignment across all distribution families (Debian, Arch, Fedora, openSUSE). All environments now consistently use the universal `BootloaderRoot` (`/tmp/coa/bootloaders`).

Packaging

- **Dependency Fixes (Arch):** Added `archiso`, `dosfstools`, and `mttools` as mandatory dependencies in the `PKGBUILD` to ensure successful live ISO generation and UEFI support.
- **Dependency Fixes (Debian):** Added `dosfstools` and `mttools` to the Debian `control` file.
- **Legacy Conflict:** Added an explicit conflict with the `penguins-eggs` package in both Debian and Arch builds to prevent overlap and ensure a clean `oa` ecosystem.

[0.6.0] - 2026-04-09

Added

- **Universal Agnostic Partitioning:** The C engine now always initializes target disks with a 3-partition GPT table (`BIOSBOOT`, `EFI`, `ROOT`), ensuring universal compatibility and disk portability regardless of the host firmware.
- **Legacy BIOS Support:** Implemented the `hatch_bios.c` action to execute `grub-install --target=i386-pc` directly onto the physical disk.
- **Dynamic Routing (Krill):** The Go orchestrator dynamically detects the presence of `/sys/firmware/efi` and injects the correct bootloader installation action (`hatch_uefi` or `hatch_bios`) into the JSON flight plan.
- **Dynamic Squashfs Detection:** The orchestrator (The Mind) automatically locates the pristine `filesystem.squashfs` image based on standard live distro mount points (e.g., `/run/live/medium/...`) and passes the exact path to the C engine.

Changed

- **Pristine Install Architecture (Unpack):** Replaced `rsync` with `unsquashfs` in `hatch_unpack.c`. The installer no longer clones the actively running (and potentially polluted) Live system, but extracts the pure factory image for a clean, pristine installation.
- **Unpack Progress Bar:** Restored the standard output of `unsquashfs` during the hatch phase to display the native progress bar to the user.
- **EFI Image Size:** Increased the `efi.img` file size for hybrid ISO generation from 4MB to 10MB to safely accommodate larger modern bootloaders.
- **FAT16 Compliance:** Forced the EFI image for hybrid boot into the `-F 16` format to satisfy strict validation checks by modern UEFI firmwares (such as OVMF on Proxmox).

Fixed

- **Hybrid ISO Boot:** Fixed UEFI boot failures on the generated ISO. Added the `-append_partition 2 0xef` flag to `xorriso` to physically inject the EFI partition into the ISO's GPT table, making it universally bootable as both a CD-ROM and a USB drive (Raw Disk).
- **Removable Media Fallback:** Corrected the UEFI bootloader path inside the ISO to strictly comply with the removable media standard (`/EFI/B00T/bootx64.efi`).
- **Partition Syncing:** Fixed a bug in `hatch_partition.c` where the BIOSBOOT partition (2MB) creation was skipped, which caused an index shift and subsequent formatting failures.

[0.5.1] - 2026-04-08

[coa] Orchestrator (The Mind)

- **New Build System:** Added `coa build` to automate C/Go compilation and native packaging (`.deb`, `PKGBUILD`).
- **Dynamic Exclusions:** Now generates runtime exclusion lists for SquashFS, moving logic away from the C engine.
- **Auto-Docs:** Integrated `coa docs` to generate man pages and shell completions (Bash, Zsh, Fish).
- **Version Controller:** Established as the Single Source of Truth for the entire project versioning.

[oa] Engine (The Body)

- **Decoupled Logic:** Removed hardcoded exclusion arrays in `lay_squash.c` in favor of dynamic `-ef` flags.
- **Version Injection:** The engine now receives its version string during compilation via Makefile macros.
- **Streamlined Execution:** Focused the C core on pure system actions, delegating policy-making to the orchestrator.

[General]

- **Git Hygiene:** Optimized `.gitignore` to keep the repository clean from build artifacts.
- **Single Source of Truth:** Centralized versioning in `build.go`.

[v0.5.0] - 2026-04-02

Added

- **Smart Surgical Unmount:** `action_cleanup` now dynamically reads `/proc/mounts` to perform an inside-out unmount (`MNT_DETACH`). It specifically targets only `liveroot` and `.overlay` branches, eliminating "zombie mounts" safely without affecting user network shares or external drives.
- **Absolute Path Support for ISO:** `action_iso` now detects if the `output_iso` parameter is an absolute path (starts with `/`). This allows the engine to write the final ISO directly to

external mounts (like a remote NAS or USB drive) bypassing local storage entirely, removing the need for `.mnt` hidden folders or symlinks.

- **Agnostic Bootloader Injection:** Introduced the `bootloaders_path` JSON parameter. `action_uefi` and `action_isolinux` can now dynamically pull bootloader binaries (like Debian's `grubx64.efi` and `isolinux.bin`) from an external path provided by the orchestrator, enabling universal booting for non-Debian host systems.
- **Hybrid UEFI/BIOS ISO Master:** `action_iso` now automatically detects the presence of UEFI payloads and dynamically generates a 4MB FAT `efi.img` (via `dd`, `mkfs.vfat`, and loop mount) to inject into `xorriso` using the `-eltorito-alt-boot` flag.
- **UEFI Trampoline Configuration:** Added a secondary `grub.cfg` inside the `EFI/BOOT/` directory to act as a trampoline. This redirects the firmware to the main `/boot/grub/grub.cfg` on the ISO9660 filesystem, completely avoiding the dreaded `grub rescue>` prompt.

Changed

- **Single Responsibility Principle Enforced:** Completely removed legacy, hardcoded cleanup routines from `action_prepare.c`. Now `prepare` only mounts, and `cleanup` only unmounts.
- **Plan JSONs Updated:** All JSON templates (`plan-standard.json`, `plan-clone.json`, `plan-crypted.json`) have been updated to include the `bootloaders_path` key and the correct modern modular actions.
- **Code Refactoring:** Removed the legacy monolithic `action_remaster` logic completely, distributing its responsibilities natively into `action_livestruct`, `action_isolinux`, and `action_uefi`.

Fixed

- **Missing GRUB Modules:** Fixed an issue in `action_uefi` where `*.mod` and `*.lst` files were not copied to the `x86_64-efi` directory, causing boot failures.

Documented

- **Universal Strategy Manifesto:** Created `docs/UNIVERSAL_STRATEGY.md` outlining the 4 pillars of the *penguins-eggs* ecosystem (Debian Passepartout, Yocto-style identity, Initramfs abstraction, and the Orchestrator role).
- **Architecture Guide Update:** Added details about the `tmpfs` Anti-Inception shield and dynamic `/home` handling to `docs/ARCHITECTURE.md`.
- **Actions Reference:** Completely overhauled `docs/ACTIONS.md` to reflect the new C engine modules (added `cleanup`, `crypted`, `scan`, `suspend`).
- **README and Roadmap:** Cleaned up `README.md`, moved future goals to a dedicated `docs/ROADMAP.md`, and added links to the philosophical architecture.

[0.2.0] - 2026-04-01

Added

- **Anti-Recursion Shield (Inception Fix):** Implemented a global `tmpfs` mask in `action_prepare.c` to hide the working directory (`LiveRoot`) from `mksquashfs` and `nftw`.

This definitively prevents infinite filesystem loops when the workspace is located inside a bind-mounted host directory (like `/home`).

- **Native Group Injection:** Added the `yocto_add_user_to_groups` helper in `oa-yocto.c` to natively append the live user to secondary groups (e.g., `sudo`, `cdrom`) directly into `/etc/group`, completely bypassing host binaries.
- **Skeleton Population:** `action_users` now correctly populates the live user's home directory by copying hidden configuration files from `/etc/skel` and applying recursive ownership.

Changed

- **Smart `/home` Handling:** Refactored `action_prepare.c` to handle the `/home` directory dynamically based on the execution `mode`. It is now mounted read-only for `clone` and `cryptoed` modes, but created as an empty directory for `standard` mode to host the newly injected live user.
- **Cleanup Fortification:** Updated `action_cleanup` to safely unmount the new anti-recursion masks and the `/home` directory using `umount2(..., MNT_DETACH)`.

Fixed

- **Missing Live Home in ISO:** Fixed a bug in `action_squash.c` where the `home/*` exclusion was aggressively deleting the freshly created live user's home during the `mksquashfs` compression in `standard` mode.

[0.1.0] - 2026-03-30

Added

- `contest.sh` create a `context.XXX.txt` to restore GEMINI contest;
- **Modular Architecture:** Decoupled core logic into `src/actions/`, making the project significantly more scalable and maintainable.
- **Execution Engine:** Implemented the `execute_verb` dispatch system in `main.c` to process the `plan.json` workflow.
- TODO: **Parameter Inheritance:** Added dual-pointer support (`cJSON *root` and `cJSON *task`) allowing actions to access both Global settings and Local task overrides.
- **Dynamic Initrd Action:** Support for command templates using `{{out}}` and `{{ver}}` placeholders for flexible initramfs generation.
- TODO: **System User Discovery:** Added `action_users` to scan for "human" users (UID ≥ 1000) using the POSIX `getpwent()` API.

Changed

- **Buffer Hardening:** Introduced `PATH_SAFE` (8192) and `CMD_MAX` (32768) constants in `oa.h` to ensure safety during complex `system()` command construction.
- **Centralized Definitions:** Consolidated all system headers and function prototypes into a single master header (`include/oa.h`).
- **Mount Fortification:** Refactored `action_prepare` to use `MS_PRIVATE` bind mounts and improved OverlayFS directory structures.

Fixed

- **Warning Purgatory:** Resolved all GCC warnings related to `-Wformat-truncation` and `-Wunused-parameter`, resulting in a "Zen" build.
 - **Logic Correction:** Fixed the Megabyte calculation in `action_scan.c` by replacing the incorrect `PATH_MAX` divisor with a proper `1024*1024` calculation.
 - **JSON Key Sincronization:** Fixed a bug in `action_iso.c` where the ISO filename and Volume ID were ignored due to a mismatch between JSON keys and C code.
-

"Code is poetry, but the changelog is the history."

blog

- <https://penguins-eggs.net>